

Project Notes 1

Problem Statement

An aviation company that provides domestic as well as international trips to the customers now wants to apply a targeted approach instead of reaching out to each of the customers. This time they want to do it digitally instead of tele-calling. Hence they have collaborated with a social networking platform, so they can learn the digital and social behavior of the customers and provide the digital advertisement on the user page of the targeted customers who have a high propensity to take up the product. Propensity of buying tickets is different for different login devices. Hence, you have to create 2 models separately for Laptop and Mobile. [Anything which is not a laptop can be considered as mobile phone usage.] The advertisements on the digital platform are bit expensive, hence you need to be very accurate while creating the models.

Importing Libraries and Checking the Data

```
In [1]: #Importing basic Libraries

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline

#to ignore warnings
import warnings
warnings.filterwarnings('ignore')
```

```
In [2]: #Loading the dataset

data = pd.read_csv(r'D:\SHUBHANK !\GL\My_Capstone_Project\Social Media Data for DSBA.cs
```

Exploring the data

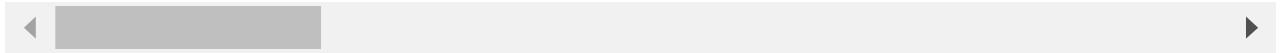
```
In [3]: #Reading the head of the dataset

data.head()
```

```
Out[3]:
```

	UserID	Taken_product	Yearly_avg_view_on_travel_page	preferred_device	total_likes_on_outstation_ch
0	1000001	Yes	307.0	iOS and Android	
1	1000002	No	367.0	iOS	
2	1000003	Yes	277.0	iOS and Android	
3	1000004	No	247.0	iOS	

	UserID	Taken_product	Yearly_avg_view_on_travel_page	preferred_device	total_likes_on_outstation_ch
4	1000005	No	202.0	iOS and Android	



In [4]:

```
#Printing the shape of the dataset

print('Total Columns: ',data.shape[1])
print('Total Rows: ',data.shape[0])
```

Total Columns: 17

Total Rows: 11760

In [5]:

```
#Let see all the names of the columns in the dataset

data.columns
```

```
Out[5]: Index(['UserID', 'Taken_product', 'Yearly_avg_view_on_travel_page',
              'preferred_device', 'total_likes_on_outstation_checkin_given',
              'yearly_avg_Outstation_checkins', 'member_in_family',
              'preferred_location_type', 'Yearly_avg_comment_on_travel_page',
              'total_likes_on_outofstation_checkin_received',
              'week_since_last_outstation_checkin', 'following_company_page',
              'montly_avg_comment_on_company_page', 'working_flag',
              'travelling_network_rating', 'Adult_flag',
              'Daily_Avg_mins_spend_on_traveling_page'],
              dtype='object')
```

Renaming few of the columns for better readability and understanding

Replacing few long words in short abbreviations, like:

1. Yearly_avg as YA
2. Daily_avg as DA
3. Preferred as Pref
4. Total_likes as TL
5. Monthly_avg as MA
6. Outstation or out of station are same, so replacing it with OS
7. Travel_page and Company_page are same, so replacing it with CP

In [6]:

```
data.rename({'Yearly_avg_view_on_travel_page': 'YA_view_on_CP',
              'preferred_device': 'pref_device',
              'total_likes_on_outstation_checkin_given': 'TL_done_on_OS_checkin',
              'yearly_avg_Outstation_checkins': 'YA_OS_checkin',
              'member_in_family': 'members_in_fam',
              'preferred_location_type': 'pref_loc_type',
              'Yearly_avg_comment_on_travel_page': 'YA_comment_on_CP',
              'total_likes_on_outofstation_checkin_received': 'TL_rec_on_OS_checkin',
              'week_since_last_outstation_checkin': 'week_since_last_OS_checkin',
              'following_company_page': 'following_CP',
              'montly_avg_comment_on_company_page': 'MA_comment_on_CP',
              'Daily_Avg_mins_spend_on_traveling_page': 'DA_mins_spend_on_CP'}, axis=1,
```

In [7]: *#Now again checking columns and shape*

```
print(data.shape)
print('\n-----')
new_column_names = data.columns
print(new_column_names)
```

(11760, 17)

```
-----
Index(['UserID', 'Taken_product', 'YA_view_on_CP', 'pref_device',
      'TL_done_on_OS_checkin', 'YA_OS_checkin', 'members_in_fam',
      'pref_loc_type', 'YA_comment_on_CP', 'TL_rec_on_OS_checkin',
      'week_since_last_OS_checkin', 'following_CP', 'MA_comment_on_CP',
      'working_flag', 'travelling_network_rating', 'Adult_flag',
      'DA_mins_spend_on_CP'],
      dtype='object')
```

In [8]: *#Size of the dataset*

```
data.size
```

Out[8]: 199920

In [9]: *#Let's Look at the basic information about the dataset*

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11760 entries, 0 to 11759
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   UserID                                11760 non-null  int64
1   Taken_product                        11760 non-null  object
2   YA_view_on_CP                        11179 non-null  float64
3   pref_device                          11707 non-null  object
4   TL_done_on_OS_checkin                11379 non-null  float64
5   YA_OS_checkin                        11685 non-null  object
6   members_in_fam                       11760 non-null  object
7   pref_loc_type                        11729 non-null  object
8   YA_comment_on_CP                    11554 non-null  float64
9   TL_rec_on_OS_checkin                11760 non-null  int64
10  week_since_last_OS_checkin           11760 non-null  int64
11  following_CP                         11657 non-null  object
12  MA_comment_on_CP                    11760 non-null  int64
13  working_flag                         11760 non-null  object
14  travelling_network_rating            11760 non-null  int64
15  Adult_flag                          11760 non-null  int64
16  DA_mins_spend_on_CP                 11760 non-null  int64
dtypes: float64(3), int64(7), object(7)
memory usage: 1.5+ MB
```

In [10]: *#Description of the data and five number summary using describe function*

```
pd.set_option('display.float_format', lambda x: '%.2f' % x) #Keeping the decimal points
data.describe().T
```

Out[10]:		count	mean	std	min	25%	50%	75%
	UserID	11760.00	1005880.50	3394.96	1000001.00	1002940.75	1005880.50	1008820.
	YA_view_on_CP	11179.00	280.83	68.18	35.00	232.00	271.00	324.
	TL_done_on_OS_checkin	11379.00	28170.48	14385.03	3570.00	16380.00	28076.00	40525.
	YA_comment_on_CP	11554.00	74.79	24.03	3.00	57.00	75.00	92.
	TL_rec_on_OS_checkin	11760.00	6531.70	4706.61	1009.00	2940.75	4948.00	8393.
	week_since_last_OS_checkin	11760.00	3.20	2.62	0.00	1.00	3.00	5.
	MA_comment_on_CP	11760.00	28.66	48.66	11.00	17.00	22.00	27.
	travelling_network_rating	11760.00	2.71	1.08	1.00	2.00	3.00	4.
	Adult_flag	11760.00	0.79	0.85	0.00	0.00	1.00	1.
	DA_mins_spend_on_CP	11760.00	13.82	9.07	0.00	8.00	12.00	18.

In [11]: *#Checking the data type for each variable in the dataset*

```
data.dtypes
```

Out[11]:

UserID	int64
Taken_product	object
YA_view_on_CP	float64
pref_device	object
TL_done_on_OS_checkin	float64
YA_OS_checkin	object
members_in_fam	object
pref_loc_type	object
YA_comment_on_CP	float64
TL_rec_on_OS_checkin	int64
week_since_last_OS_checkin	int64
following_CP	object
MA_comment_on_CP	int64
working_flag	object
travelling_network_rating	int64
Adult_flag	int64
DA_mins_spend_on_CP	int64
dtype:	object

In [12]: *#Any null or missing values in dataset ?? Let's check*

```
data.isnull().any()
```

Out[12]:

UserID	False
Taken_product	False
YA_view_on_CP	True
pref_device	True
TL_done_on_OS_checkin	True
YA_OS_checkin	True
members_in_fam	False
pref_loc_type	True
YA_comment_on_CP	True

TL_rec_on_OS_checkin	False
week_since_last_OS_checkin	False
following_CP	True
MA_comment_on_CP	False
working_flag	False
travelling_network_rating	False
Adult_flag	False
DA_mins_spend_on_CP	False
dtype:	bool

Boolean value False means there is no missing data in the column and True means there are some missing values in that column.

```
In [13]: #Yes, there are missing values in the dataset.

#Let's check how many missing values are there in those columns

data.isnull().sum()
```

```
Out[13]: UserID                0
Taken_product                0
YA_view_on_CP              581
pref_device                 53
TL_done_on_OS_checkin      381
YA_OS_checkin              75
members_in_fam              0
pref_loc_type               31
YA_comment_on_CP          206
TL_rec_on_OS_checkin        0
week_since_last_OS_checkin  0
following_CP               103
MA_comment_on_CP            0
working_flag                0
travelling_network_rating   0
Adult_flag                  0
DA_mins_spend_on_CP         0
dtype: int64
```

```
In [14]: #Total count of the missing values

print('Total missing values:', data.isnull().sum().sum())
```

Total missing values: 1430

Data Cleaning

Dropping 2 columns

```
In [15]: #First of all dropping the UserID column as it is not so useful for our analysis
#Also we are dropping montly_avg_comment_on_company_page column bcause we already have
#it does not make sense to keep both columns as it is indicating same action

data.drop(['UserID', 'MA_comment_on_CP'], axis = 1, inplace = True)
```

Treating Missing Values

```
In [16]: #While checking, we've got 1430 total missing values in the dataset, which is around 12%
#So we decide not to do any random imputation because it may affect our model

#Lets just drop them all

data.dropna(inplace=True)
```

```
In [17]: #After dropping, Let's check the null values again

data.isnull().sum()
```

```
Out[17]: Taken_product          0
YA_view_on_CP          0
pref_device            0
TL_done_on_OS_checkin  0
YA_OS_checkin          0
members_in_fam         0
pref_loc_type          0
YA_comment_on_CP       0
TL_rec_on_OS_checkin   0
week_since_last_OS_checkin  0
following_CP           0
working_flag           0
travelling_network_rating  0
Adult_flag            0
DA_mins_spend_on_CP    0
dtype: int64
```

Now all the null values are dropped from the dataset.

```
In [18]: #Let's check the shape and info again

print('Total Columns: ',data.shape[1])
print('Total Rows: ',data.shape[0])
print('\n-----')
data.info()
```

```
Total Columns:  15
Total Rows:  10456
```

```
-----
<class 'pandas.core.frame.DataFrame'>
Int64Index: 10456 entries, 0 to 11759
Data columns (total 15 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   Taken_product         10456 non-null  object
 1   YA_view_on_CP         10456 non-null  float64
 2   pref_device           10456 non-null  object
 3   TL_done_on_OS_checkin 10456 non-null  float64
 4   YA_OS_checkin         10456 non-null  object
 5   members_in_fam        10456 non-null  object
 6   pref_loc_type         10456 non-null  object
 7   YA_comment_on_CP      10456 non-null  float64
 8   TL_rec_on_OS_checkin  10456 non-null  int64
 9   week_since_last_OS_checkin 10456 non-null  int64
10   following_CP          10456 non-null  object
11   working_flag          10456 non-null  object
```

```

12 travelling_network_rating    10456 non-null    int64
13 Adult_flag                    10456 non-null    int64
14 DA_mins_spend_on_CP          10456 non-null    int64
dtypes: float64(3), int64(5), object(7)
memory usage: 1.3+ MB

```

Now there are 10456 records and no null/missing values present.

Dropping the rows which has special characters

While checking the dataset, we found out that there is a special character in the column 'YA_OS_checkin'. So let's drop the rows with special character.

```

In [19]: #First checking the row with special character in the column
data[data.YA_OS_checkin.str.contains(r'[@#&$%+~/*'])

#dropping the complete row
data.drop(data[data.YA_OS_checkin.str.contains(r'^0-9a-zA-Z'])).index, inplace=True)

```

```

In [20]: #reprinting the shape now

data.shape

```

```
Out[20]: (10455, 15)
```

Now we are left with have 10455 rows and 15 columns.

Let's do the Univariate Analysis

```

In [21]: #Taken_product (Whether the user has taken their product or not)

print(data.Taken_product.value_counts())
print('\n-----')
print('Proportion:\n',data.Taken_product.value_counts(normalize=True))

```

```

No      8762
Yes     1693
Name: Taken_product, dtype: int64

```

```

-----
Proportion:
No      0.84
Yes     0.16
Name: Taken_product, dtype: float64

```

84% people did not take the product. This means that data is not balanced. We will deal with problem later.

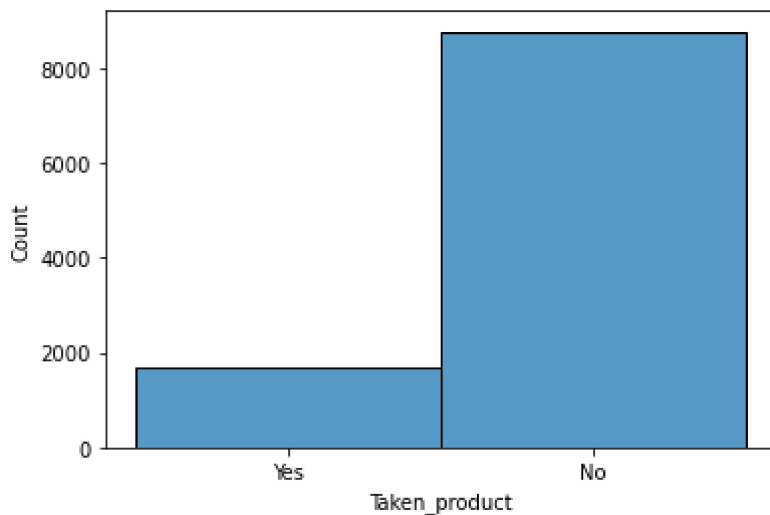
```

In [22]: #Histogram for Taken_product

sns.histplot(data=data, x="Taken_product")

```

```
Out[22]: <AxesSubplot:xlabel='Taken_product', ylabel='Count'>
```



In [23]:

```
#Pref_device (Preferred device of the user through which user do Login)

print(data.pref_device.value_counts())
print('\n-----')
print('Proportion: \n',data.pref_device.value_counts(normalize=True))
```

```
Tab          4172
iOS and Android 3246
Laptop       1108
iOS          859
Mobile       600
Android      244
Android OS   126
ANDROID      97
Other         2
Others        1
Name: pref_device, dtype: int64
```

```
-----
Proportion:
Tab          0.40
iOS and Android 0.31
Laptop       0.11
iOS          0.08
Mobile       0.06
Android      0.02
Android OS   0.01
ANDROID      0.01
Other        0.00
Others       0.00
Name: pref_device, dtype: float64
```

We can see there are so many devices mentioned in the dataset. But as per the information provided in the problem statement that 'Anything which is not a laptop can be considered as mobile phone usage', so we will replace everything with mobile except Laptop.

In [24]:

```
data['pref_device'].replace({'Tab': 'Mobile', 'iOS and Android': 'Mobile', 'iOS': 'Mobi
                             'Android': 'Mobile', 'Android OS': 'Mobile', 'ANDROID': 'Mo
                             'iOS and Android': 'Mobile', 'Other': 'Mobile', 'Others': '

```

```
In [25]: #Now again checking pref_device column

print(data.pref_device.value_counts())
print('\n-----')
print('Proportion: \n',data.pref_device.value_counts(normalize=True))
```

```
Mobile    9347
Laptop    1108
Name: pref_device, dtype: int64
```

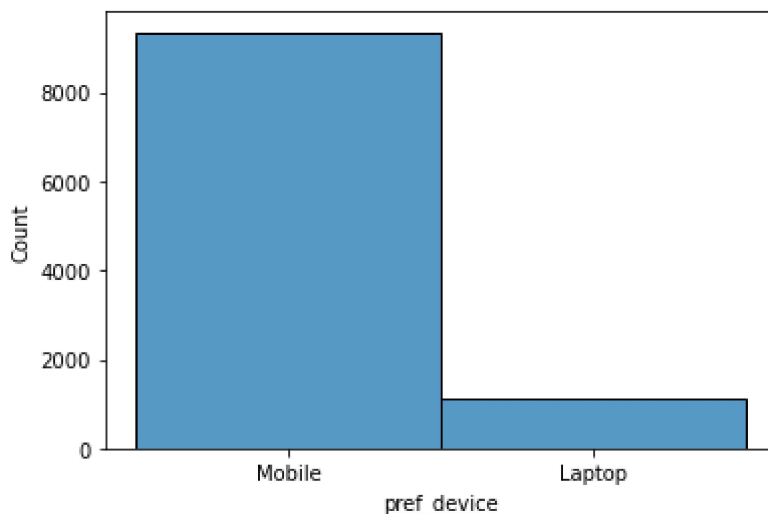
```
-----
Proportion:
Mobile    0.89
Laptop    0.11
Name: pref_device, dtype: float64
```

This indicates that 89% of the people in our data use Mobile as their preferred device.

```
In [26]: #Histogram for pref_device

sns.histplot(data=data, x="pref_device")
```

```
Out[26]: <AxesSubplot:xlabel='pref_device', ylabel='Count'>
```



```
In [27]: #Pref_Loc_type (Preferred Location type of the user)

print(data.pref_loc_type.value_counts())
```

```
Beach          2424
Financial      1875
Historical site 1856
Medical        1448
Big Cities     636
Other          581
Trekking       528
Social media   506
Entertainment  383
Hill Stations  108
Tour Travel    53
Tour and Travel 37
Game           10
```

```

OTT          6
Movie        4
Name: pref_loc_type, dtype: int64

```

We can see that there are various preferred location type available in the data. Let's assign few of them into one category and organize these.

1. We will take Beach, Historical site, Trekking, Tour Travel, Tour and Travel and Hill Stations in one category and will name it 'Adventure and Tourism'.
2. We will be replacing Game, OTT, Movie and Social Media with one already available category Entertainment.

```

In [28]: data['pref_loc_type'].replace({'Beach': 'Adventure and Tourism', 'Historical site': 'Adv
        'Trekking': 'Adventure and Tourism', 'Tour Travel': 'Ad
        'Tour and Travel': 'Adventure and Tourism', 'Hill Statio
        'Game': 'Entertainment', 'OTT': 'Entertainment', 'Movie'
        'Social media': 'Entertainment'}, inplace=True)

```

```

In [29]: #Checking the updated values and checking the proportion for each

print(data.pref_loc_type.value_counts())
print('\n-----')
print('Proportion: \n', data.pref_loc_type.value_counts(normalize=True))

```

```

Adventure and Tourism    5006
Financial                1875
Medical                  1448
Entertainment            909
Big Cities                636
Other                    581
Name: pref_loc_type, dtype: int64

-----
Proportion:
Adventure and Tourism    0.48
Financial                 0.18
Medical                  0.14
Entertainment            0.09
Big Cities                0.06
Other                    0.06
Name: pref_loc_type, dtype: float64

```

So after organizing the categories, we found out that 'Adventure and Tourism' is the most preferred location type with 48%. Then 18% of the user has 'Financial' as their preferred location type, because they might travel mostly for business purpose. Then comes the 'Medical' as preferred location, as 14% of the user travel for their medical purposes. Only 9% of the users has 'Entertainment' as their preferred location type.

```

In [30]: #Histogram for pref_loc_type

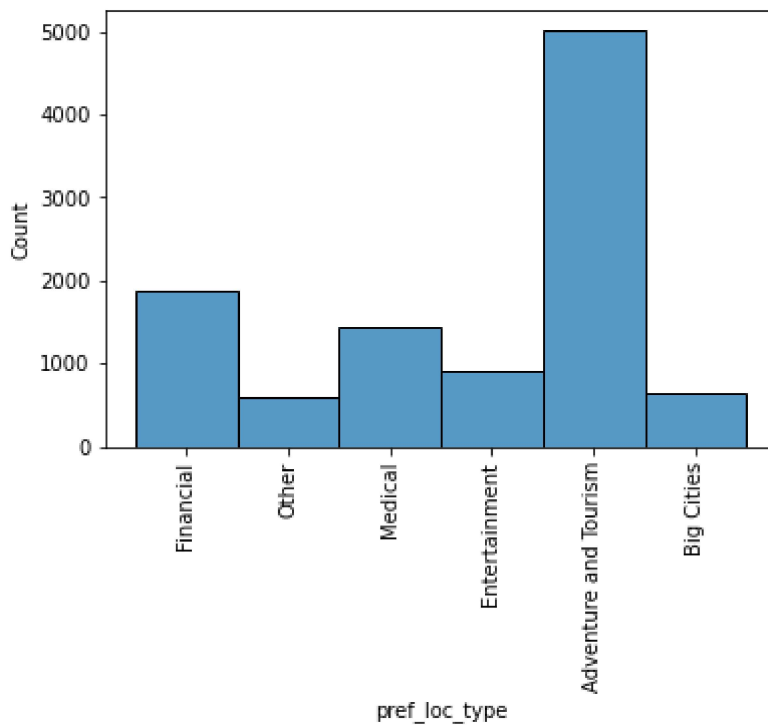
plt.xticks(rotation=90)
sns.histplot(data=data, x="pref_loc_type")

```

```

Out[30]: <AxesSubplot:xlabel='pref_loc_type', ylabel='Count'>

```



```
In [31]: #Following_CP (whether the user is following the company page or not)
print(data.following_CP.value_counts())
```

```
No      7517
Yes      2923
1         10
0          5
Name: following_CP, dtype: int64
```

Here we need the data in the form of 'Yes' and 'No'. So we need to replace numeric value of 1 to Yes and 0 to No.

```
In [32]: #replacing 1 and 0 with Yes and No respectively.
data['following_CP'].replace({'1': 'Yes', '0': 'No'}, inplace=True)
```

```
In [33]: #Again checking
print(data.following_CP.value_counts())
print('\n-----')
print('Proportion: \n',data.following_CP.value_counts(normalize = True))
```

```
No      7522
Yes      2933
Name: following_CP, dtype: int64

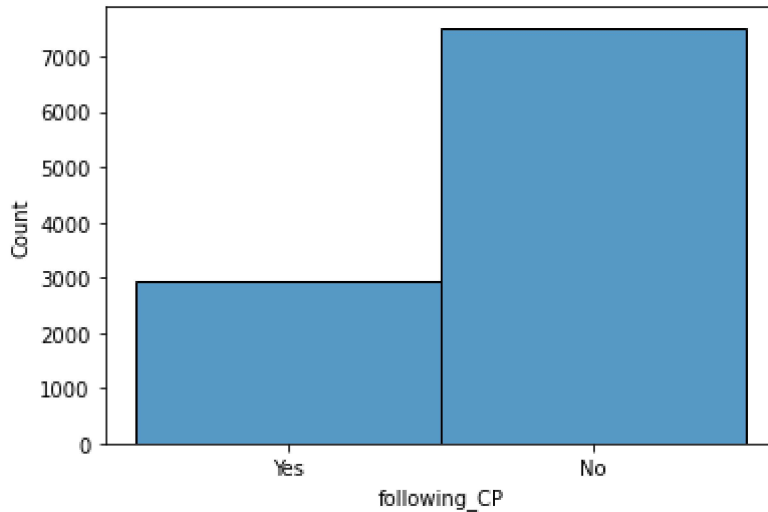
-----
Proportion:
No      0.72
Yes     0.28
Name: following_CP, dtype: float64
```


We see that only 28% of the users are following the company page and 72% still do not follow.

```
In [34]: #Histogram for following_CP

sns.histplot(data=data, x="following_CP")
```

```
Out[34]: <AxesSubplot:xlabel='following_CP', ylabel='Count'>
```



```
In [35]: #Working_flag (whether the user is working somewhere or not)

print(data.working_flag.value_counts())
print('\n-----')
print('Proportion: \n',data.working_flag.value_counts(normalize = True))
```

```
No      8846
Yes      1609
Name: working_flag, dtype: int64
```

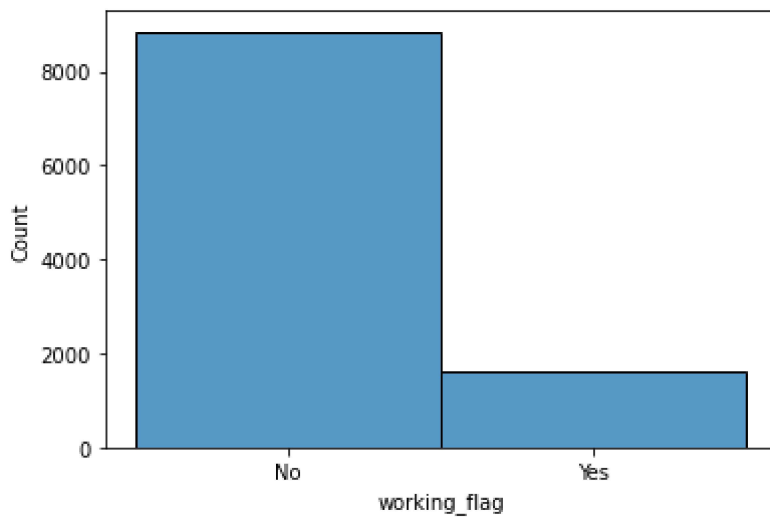
```
-----
Proportion:
No      0.85
Yes      0.15
Name: working_flag, dtype: float64
```

85% of the users do not work anywhere. Only 15% users are working.

```
In [36]: #Histogram for working_flag

sns.histplot(data=data, x="working_flag")
```

```
Out[36]: <AxesSubplot:xlabel='working_flag', ylabel='Count'>
```



```
In [37]: #members_in_fam (Number of members in the family of the user)

data.members_in_fam.value_counts()
```

```
Out[37]: 3      4087
         4      2843
         2      1983
         1      1192
         5       331
        10        10
        Three         9
        Name: members_in_fam, dtype: int64
```

Instead of numeric 3, it is in alphabet form, let's replace/correct it.

```
In [38]: data['members_in_fam'].replace({'Three': '3'}, inplace=True)
```

```
In [39]: data.members_in_fam.value_counts()
```

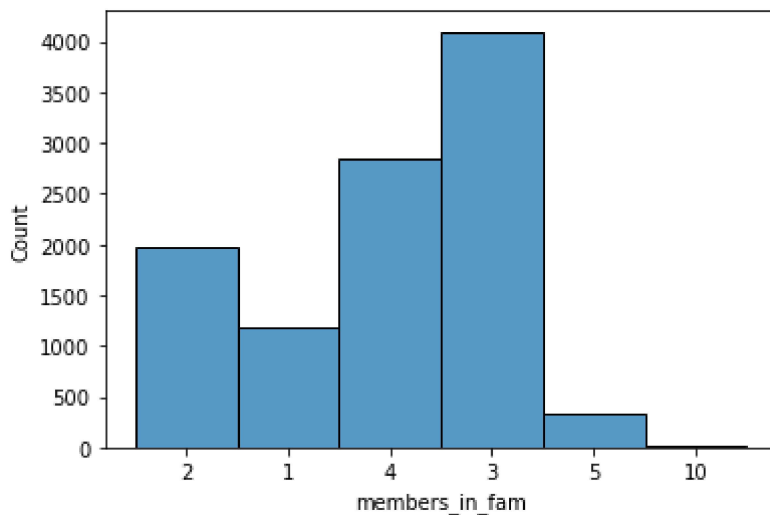
```
Out[39]: 3      4096
         4      2843
         2      1983
         1      1192
         5       331
        10        10
        Name: members_in_fam, dtype: int64
```

Most of the users have 3 members in their family.

```
In [40]: #Histogram for members_in_fam

sns.histplot(data=data, x="members_in_fam")
```

```
Out[40]: <AxesSubplot:xlabel='members_in_fam', ylabel='Count'>
```



```
In [41]: #Adult_flag (Whether the user is Adult or not)
#it should be either only 0 and 1, where 0 means user is not adult and 1 means user is

data.Adult_flag.value_counts()
```

```
Out[41]: 0    4476
         1    4248
         2    1114
         3     617
         Name: Adult_flag, dtype: int64
```

```
In [42]: #here we will replace 2 and 3 with 1 because it's a categorical variable which must hav
#User can either be adult or not, so it should have values of either yes and no, or 1 a

data['Adult_flag'].replace({2: 1, 3: 1}, inplace=True)
```

```
In [43]: #Now checking again

data.Adult_flag.value_counts()
```

```
Out[43]: 1    5979
         0    4476
         Name: Adult_flag, dtype: int64
```

```
In [44]: data.Adult_flag.value_counts(normalize=True)
```

```
Out[44]: 1    0.57
         0    0.43
         Name: Adult_flag, dtype: float64
```

Around 57% users are adult.

Correcting the data types of few of the data columns

As we have checked that few variables indicating the incorrect data types like:

1. 'YA_view_on_CP' should have integer type instead of float type.
2. 'TL_done_on_OS_checkin' should have integer type instead of float type.
3. 'YA_OS_checkin' should have integer type instead of object type.

4. 'members_in_fam' should have integer type instead of object type.

5. 'YA_comment_on_CP' should have integer type instead of float type.

```
In [45]: data['YA_view_on_CP'] = data['YA_view_on_CP'].astype(np.int64)
data['TL_done_on_OS_checkin'] = data['TL_done_on_OS_checkin'].astype(np.int64)
data['YA_OS_checkin'] = data['YA_OS_checkin'].astype(np.int64)
data['members_in_fam'] = data['members_in_fam'].astype(np.int64)
data['YA_comment_on_CP'] = data['YA_comment_on_CP'].astype(np.int64)
```

```
In [46]: #Let's check the dtypes again

data.dtypes
```

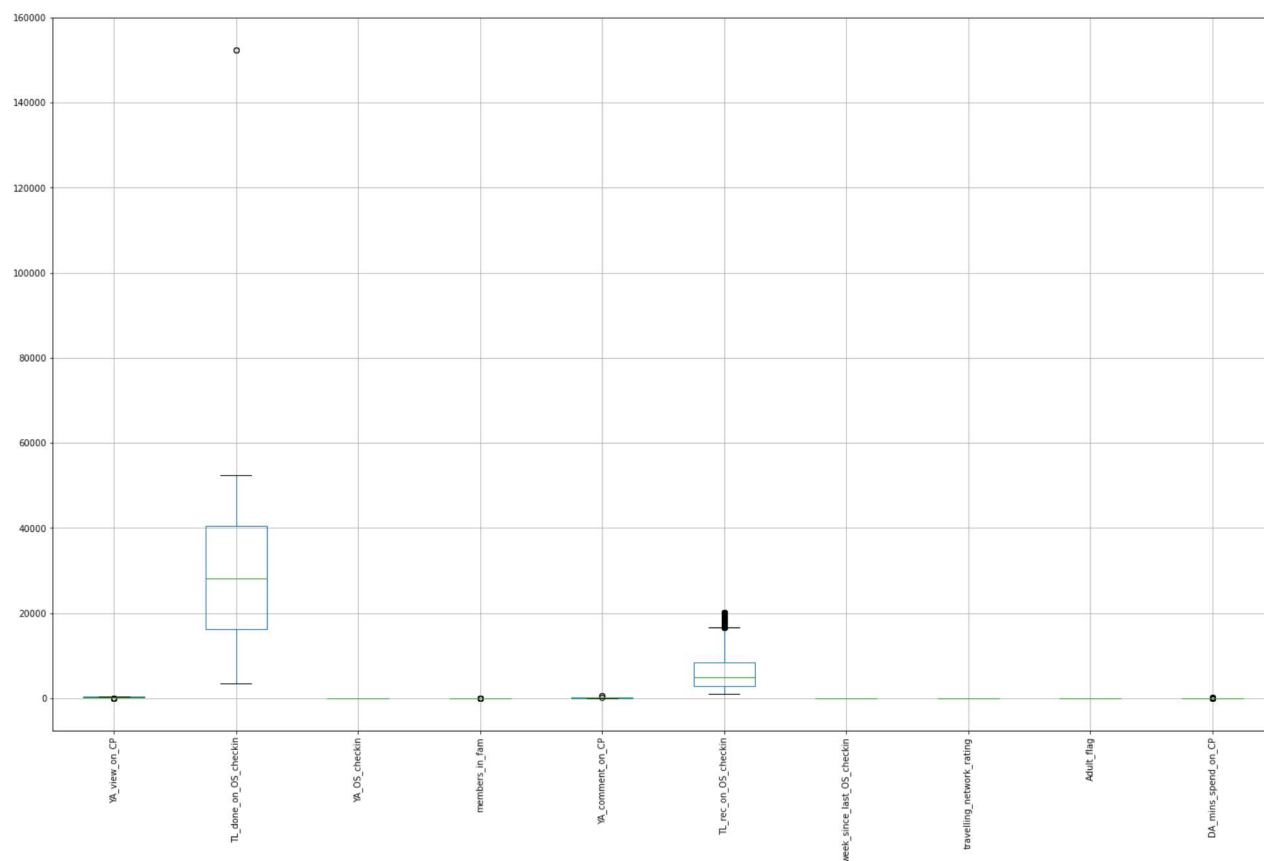
```
Out[46]: Taken_product          object
YA_view_on_CP              int64
pref_device               object
TL_done_on_OS_checkin     int64
YA_OS_checkin             int64
members_in_fam            int64
pref_loc_type             object
YA_comment_on_CP          int64
TL_rec_on_OS_checkin      int64
week_since_last_OS_checkin int64
following_CP              object
working_flag              object
travelling_network_rating int64
Adult_flag                int64
DA_mins_spend_on_CP       int64
dtype: object
```

Detecting and Treating the Outliers

```
In [47]: #Detecting Outliers by making a box plot for numerical attributes

data.boxplot(figsize=(25, 15), rot=90)
```

```
Out[47]: <AxesSubplot:>
```



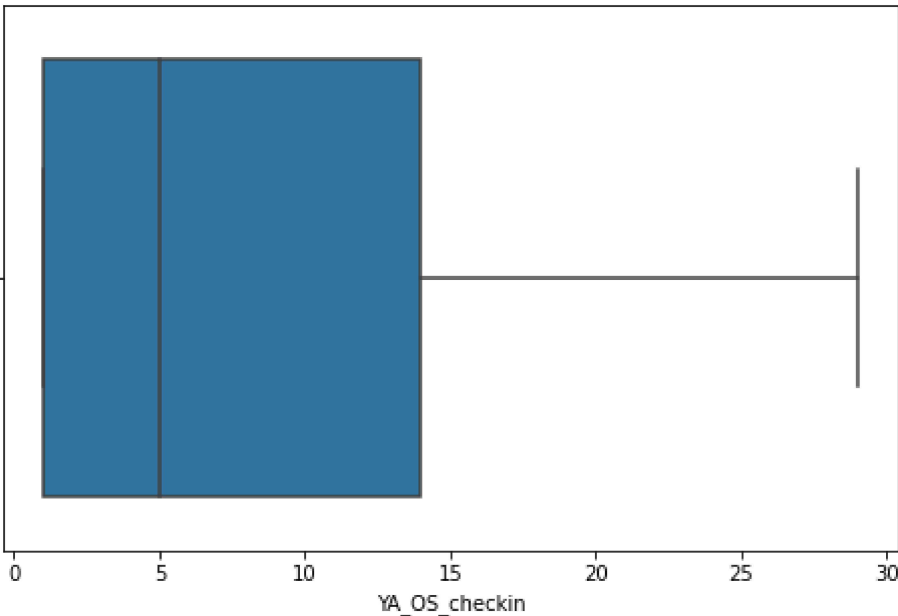
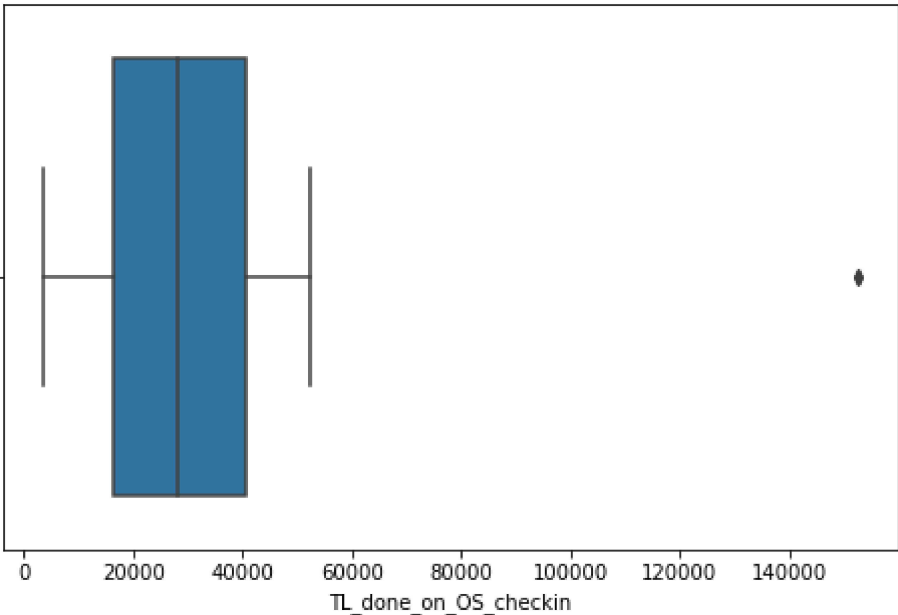
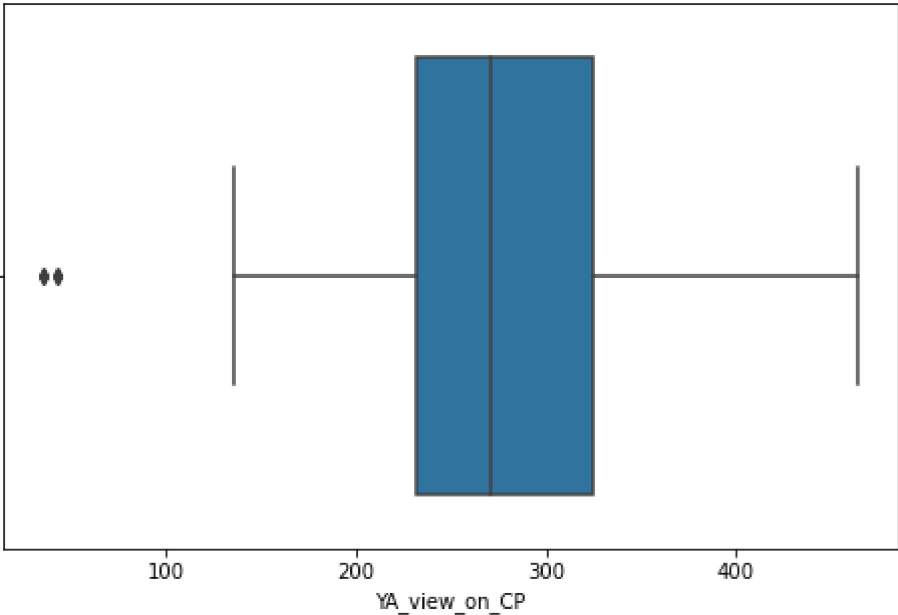
There are few outliers in the data.

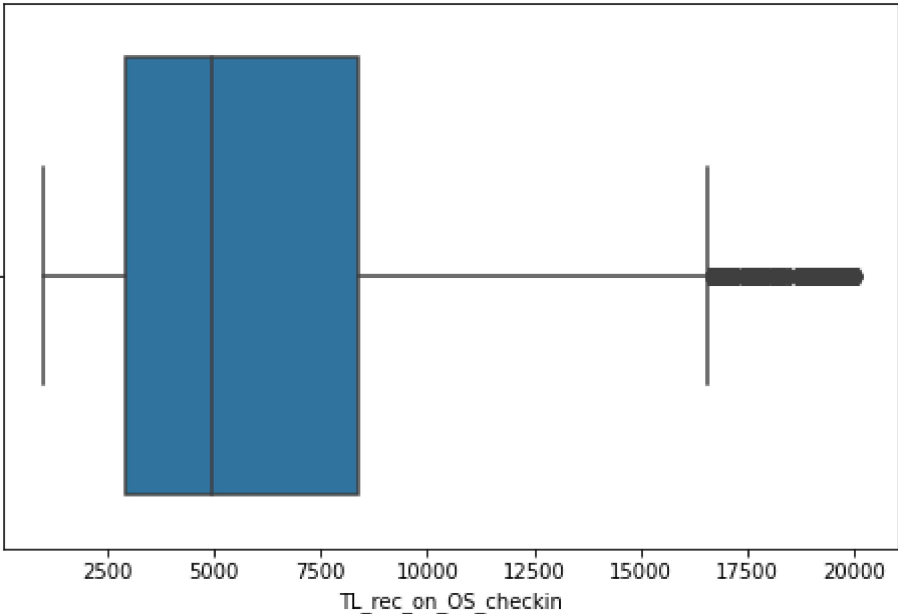
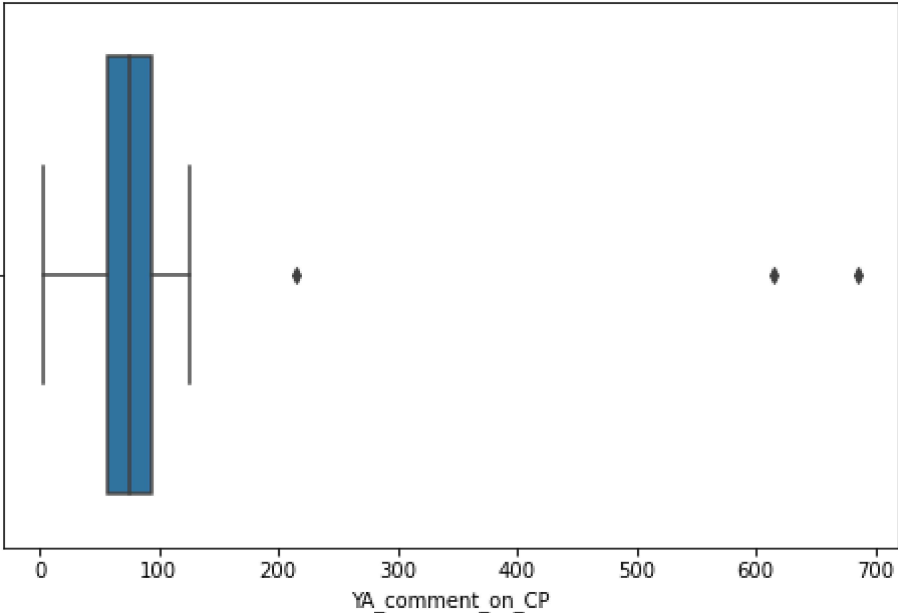
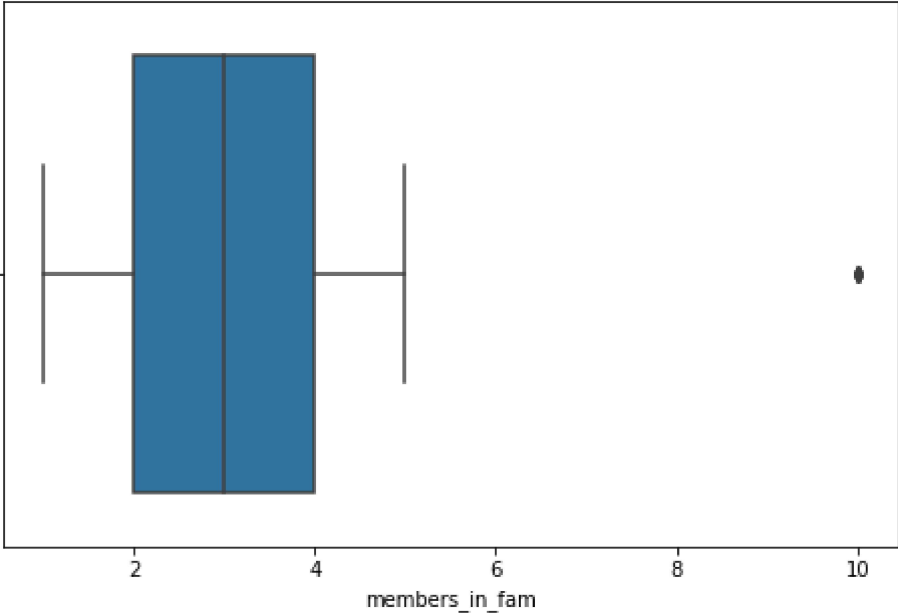
In [48]:

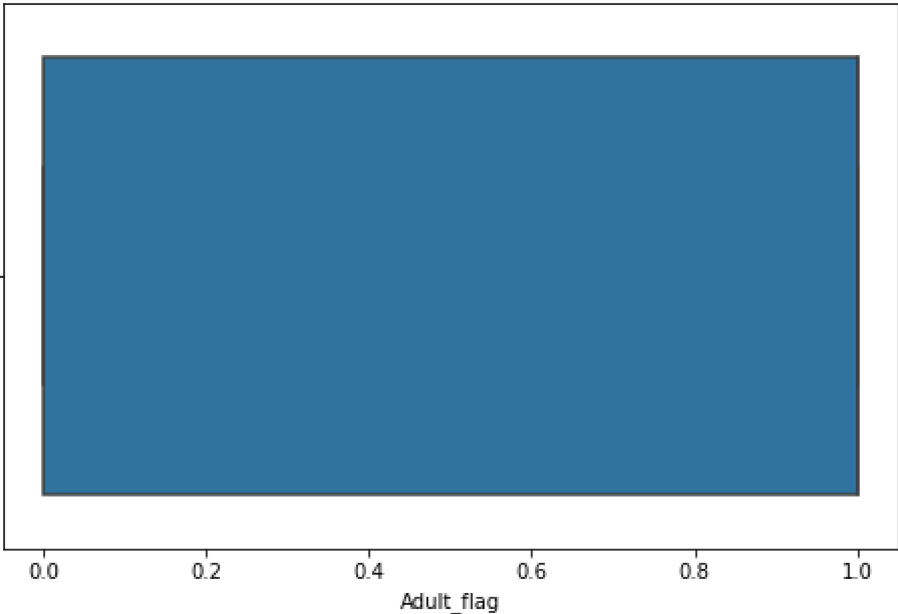
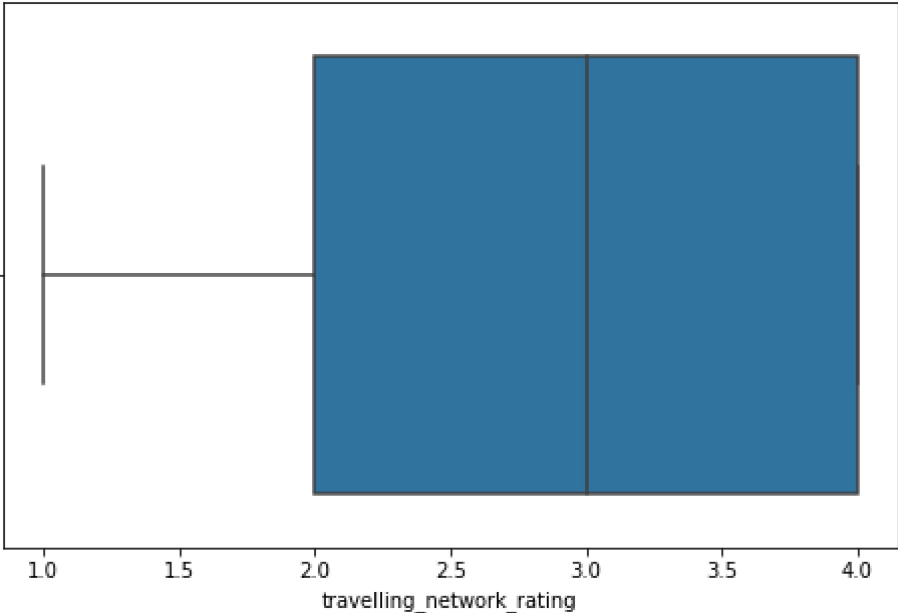
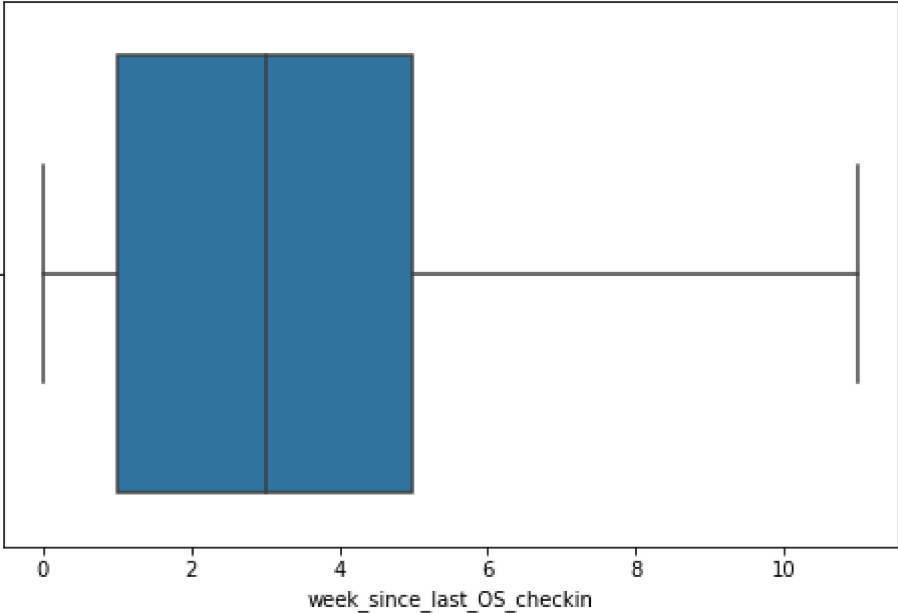
```
#Let's look at them one by one for each numerical variables

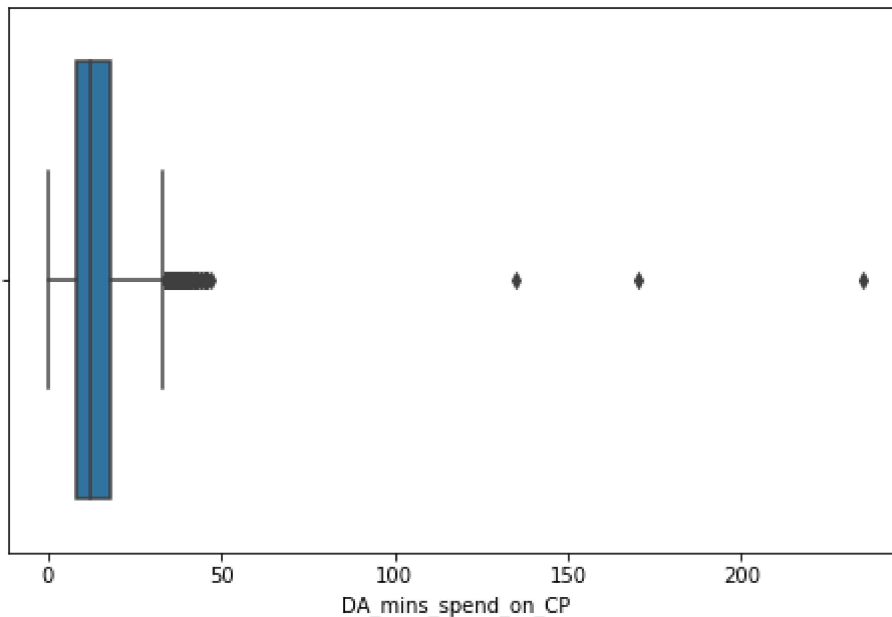
num_columns = ['YA_view_on_CP', 'TL_done_on_OS_checkin', 'YA_OS_checkin', 'members_in_fam',
               'TL_rec_on_OS_checkin', 'week_since_last_OS_checkin', 'travelling_network_rating',
               'DA_mins_spend_on_CP']

for col in num_columns:
    plt.figure(figsize=(8,5))
    sns.boxplot(data[col])
    plt.show()
```









Treating Outliers

Treating the outliers by using Inter Quartile Range (IQR) method. This technique uses the IQR scores to remove outliers. The rule of thumb is that anything not in the range of $(Q1 - 1.5 \text{ IQR})$ and $(Q3 + 1.5 \text{ IQR})$ is an outlier, and can be removed.

```
In [50]: #Calculating Q1 and Q3 values and assigning the upper limit (UL) and Lower Limit (LL)

Q1 = data[num_columns].quantile(0.25)
Q3 = data[num_columns].quantile(0.75)
IQR = Q3 - Q1
UL = Q3 + 1.5*IQR
LL = Q1 - 1.5*IQR
```

```
In [52]: #Sum of outliers in each column
pd.options.display.max_rows = None
((data[num_columns] > UL) | (data[num_columns] < LL)).sum()
```

```
Out[52]: YA_view_on_CP          8
         TL_done_on_OS_checkin  2
         YA_OS_checkin         0
         members_in_fam       10
         YA_comment_on_CP      3
         TL_rec_on_OS_checkin 818
         week_since_last_OS_checkin 0
         travelling_network_rating 0
         Adult_flag           0
         DA_mins_spend_on_CP    315
         dtype: int64
```

```
In [54]: #Total outliers

print('Total number of outliers in the dataset:', ((data[num_columns] > UL) | (data[num_

Total number of outliers in the dataset: 1156
```

It is almost around 10%, we will remove those from our dataset

We will be defining a function named treat_outliers using that function we will remove those outliers.

```
In [56]: def treat_outliers(col):
sorted(col)
Q1,Q3=np.percentile(col,[25,75])
IQR=Q3-Q1
lower_range= Q1-(1.5 * IQR)
upper_range= Q3+(1.5 * IQR)
return lower_range, upper_range

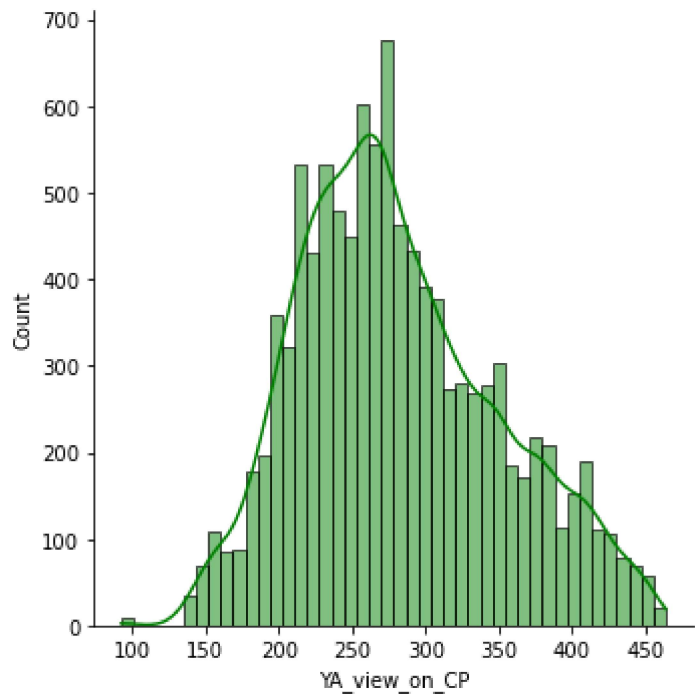
for column in data[num_columns].columns:
lr,ur=treat_outliers(data[column])
data[column]=np.where(data[column]>ur,ur,data[column])
data[column]=np.where(data[column]<lr,lr,data[column])
```

```
In [57]: #Now checking again.
((data[num_columns] > UL) | (data[num_columns] < LL)).sum()
```

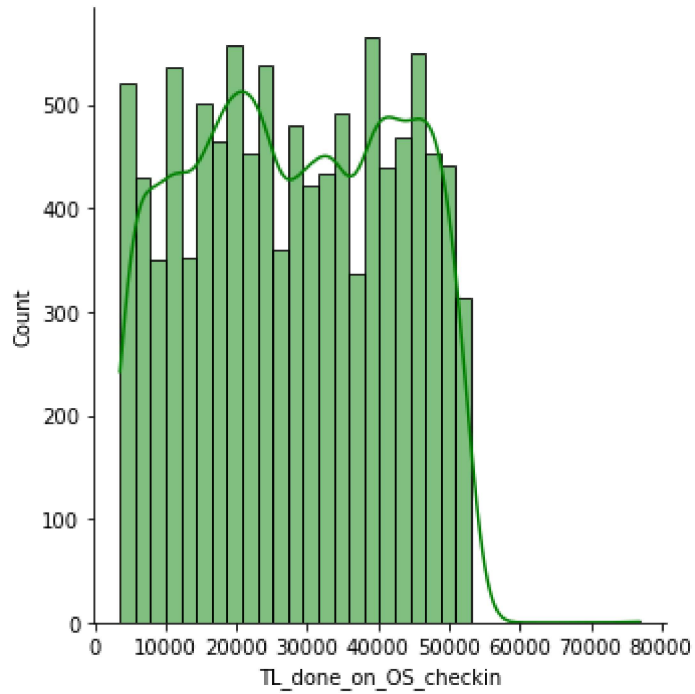
```
Out[57]: YA_view_on_CP          0
TL_done_on_OS_checkin        0
YA_OS_checkin                0
members_in_fam               0
YA_comment_on_CP             0
TL_rec_on_OS_checkin         0
week_since_last_OS_checkin   0
travelling_network_rating    0
Adult_flag                   0
DA_mins_spend_on_CP          0
dtype: int64
```

```
In [64]: for x in num_columns:
plt.figure(figsize=(6,4))
sns.displot(data[x], kde=True, color='green')
plt.show()
```

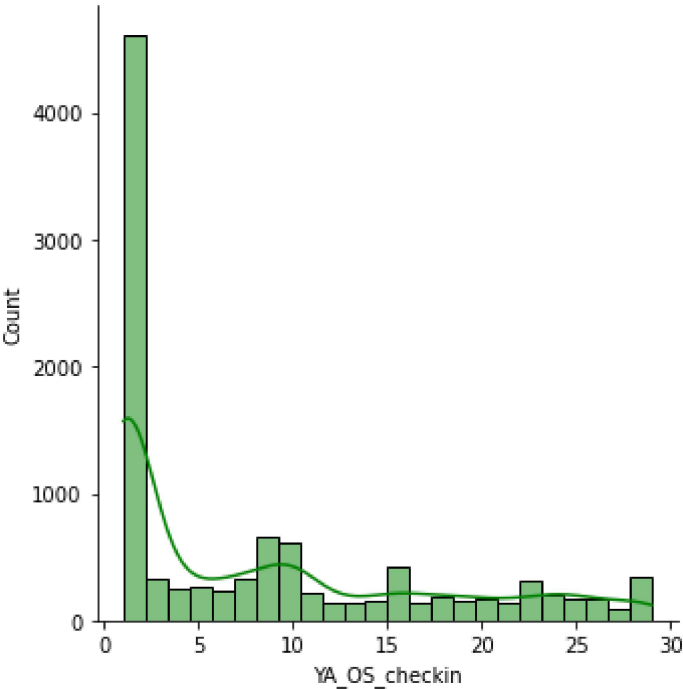
<Figure size 432x288 with 0 Axes>



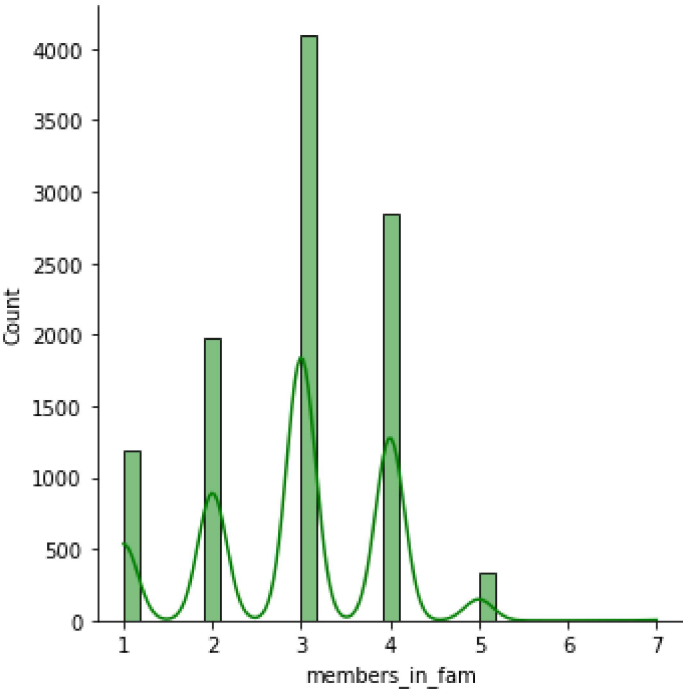
<Figure size 432x288 with 0 Axes>



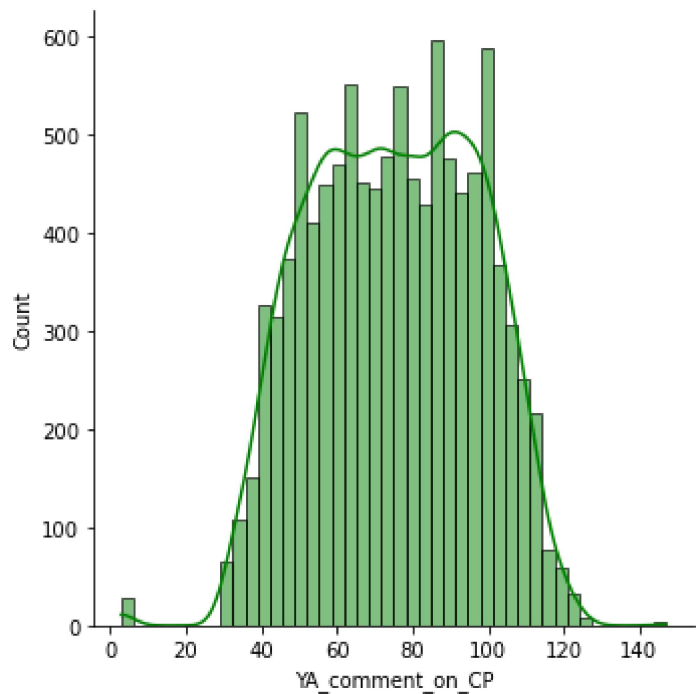
<Figure size 432x288 with 0 Axes>



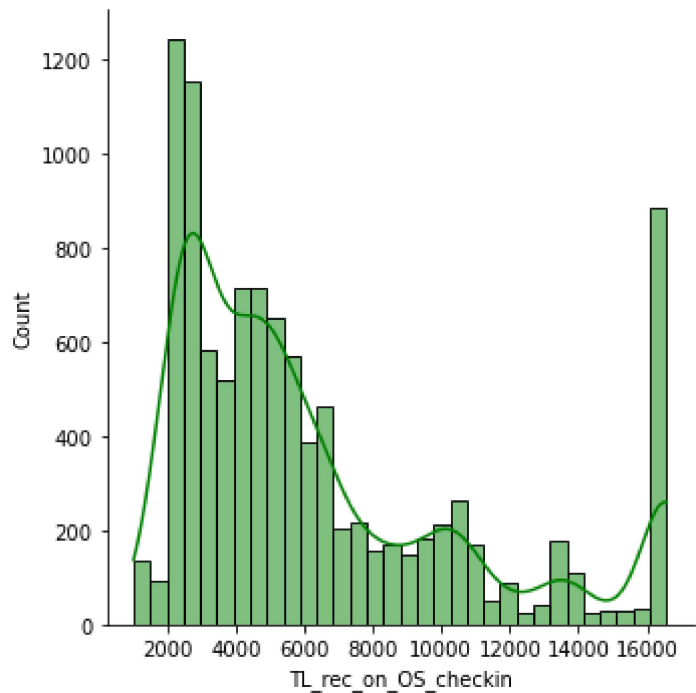
<Figure size 432x288 with 0 Axes>



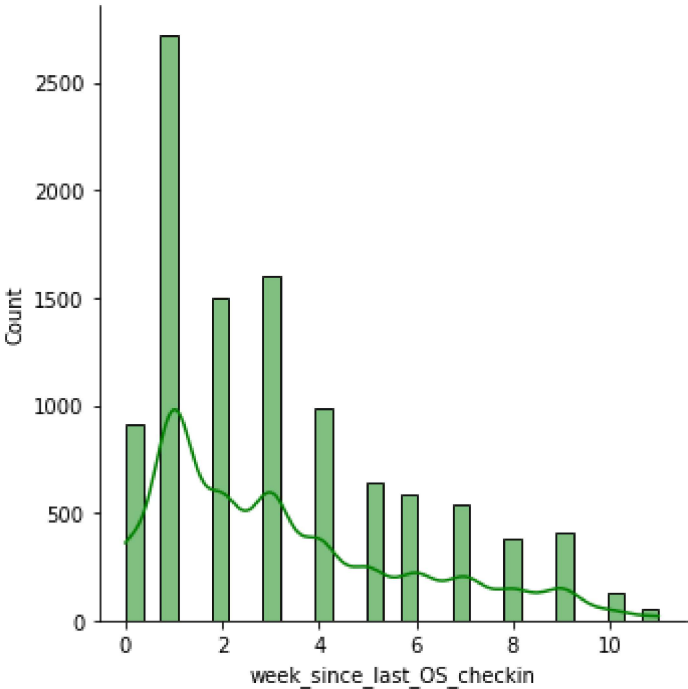
<Figure size 432x288 with 0 Axes>



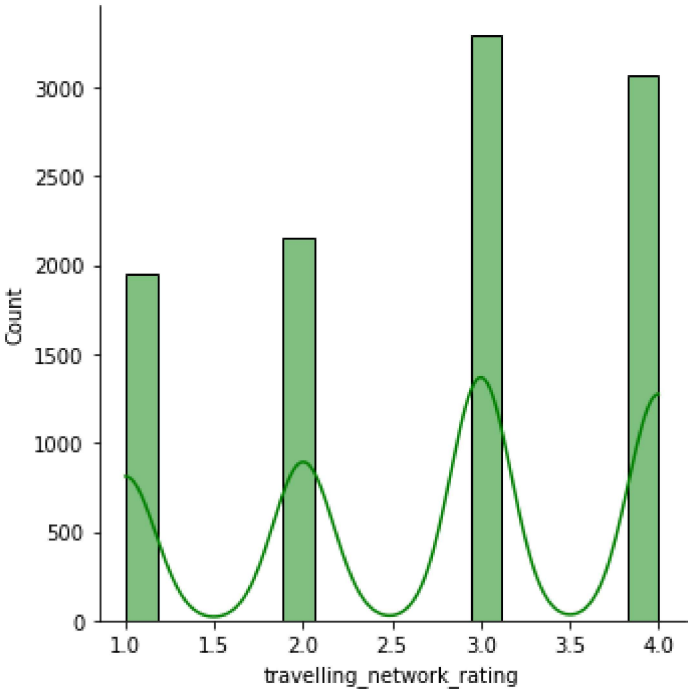
<Figure size 432x288 with 0 Axes>



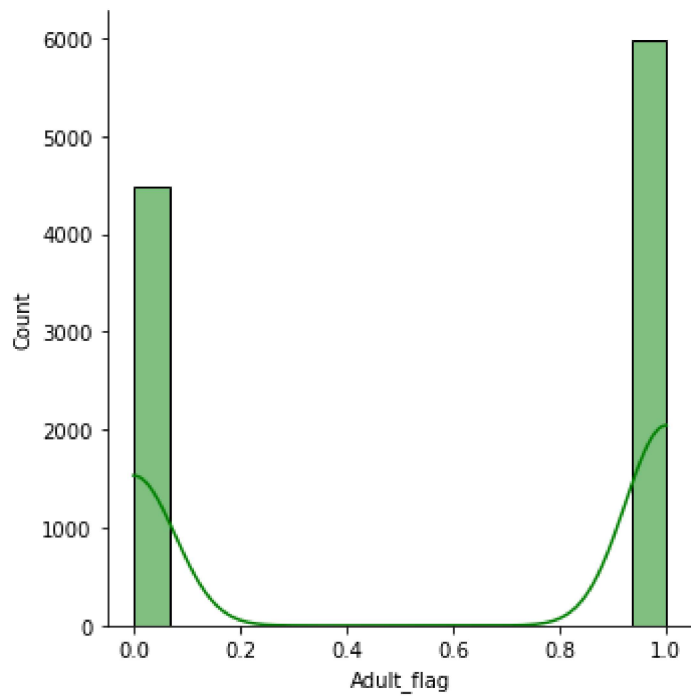
<Figure size 432x288 with 0 Axes>



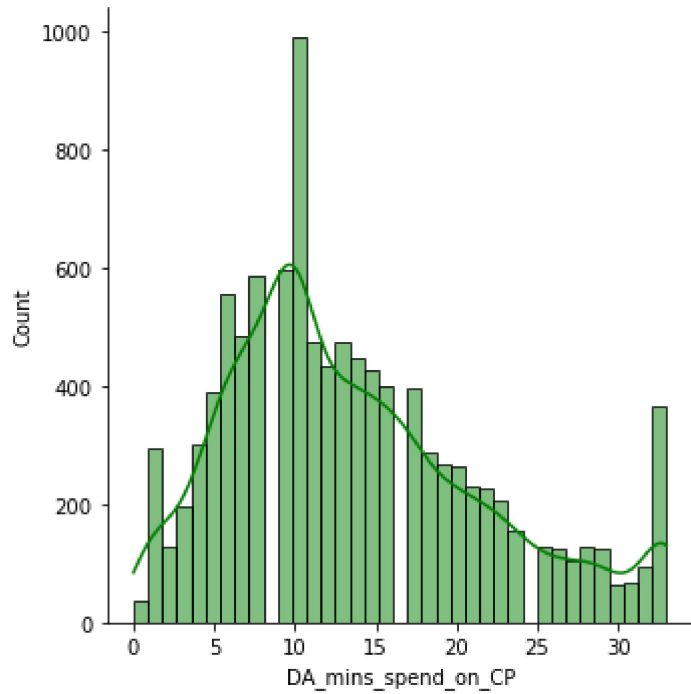
<Figure size 432x288 with 0 Axes>



<Figure size 432x288 with 0 Axes>



<Figure size 432x288 with 0 Axes>



Bivariate Analysis

```
In [65]: #Now let see the correlation between the variables using correlation matrices

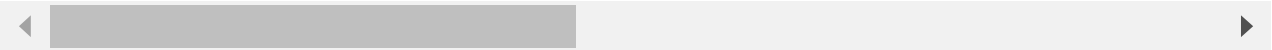
corr = data.corr()

corr
```

```
Out[65]:
```

	YA_view_on_CP	TL_done_on_OS_checkin	YA_OS_checkin	members_in_fam	YA...
YA_view_on_CP	1.00	0.01	0.01	0.20	
TL_done_on_OS_checkin	0.01	1.00	0.00	-0.01	

	YA_view_on_CP	TL_done_on_OS_checkin	YA_OS_checkin	members_in_fam	YA
YA_OS_checkin	0.01	0.00	1.00	0.02	
members_in_fam	0.20	-0.01	0.02	1.00	
YA_comment_on_CP	0.04	0.01	0.06	0.01	
TL_rec_on_OS_checkin	0.50	0.01	-0.00	0.10	
week_since_last_OS_checkin	0.29	0.04	-0.03	0.11	
travelling_network_rating	0.05	0.00	0.01	-0.01	
Adult_flag	0.09	0.03	0.04	0.01	
DA_mins_spend_on_CP	0.64	0.02	0.01	0.13	



In [69]:

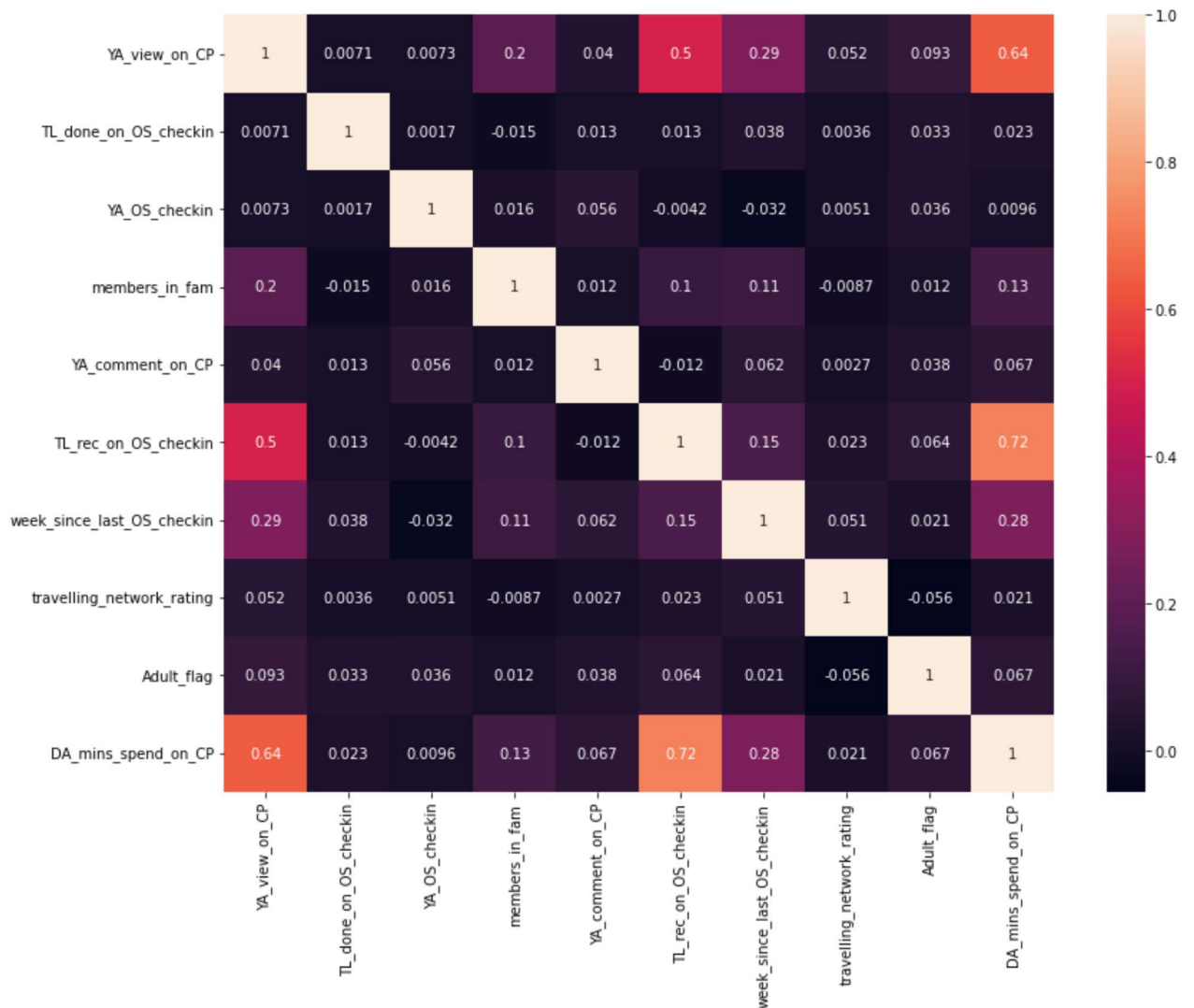
```
#Heatmap

#Now we will plot the heatmap to display the correlation between the variables:

fig, ax = plt.subplots(figsize=(13,10))
sns.heatmap(corr,annot=True)
```

Out[69]:

<AxesSubplot:>



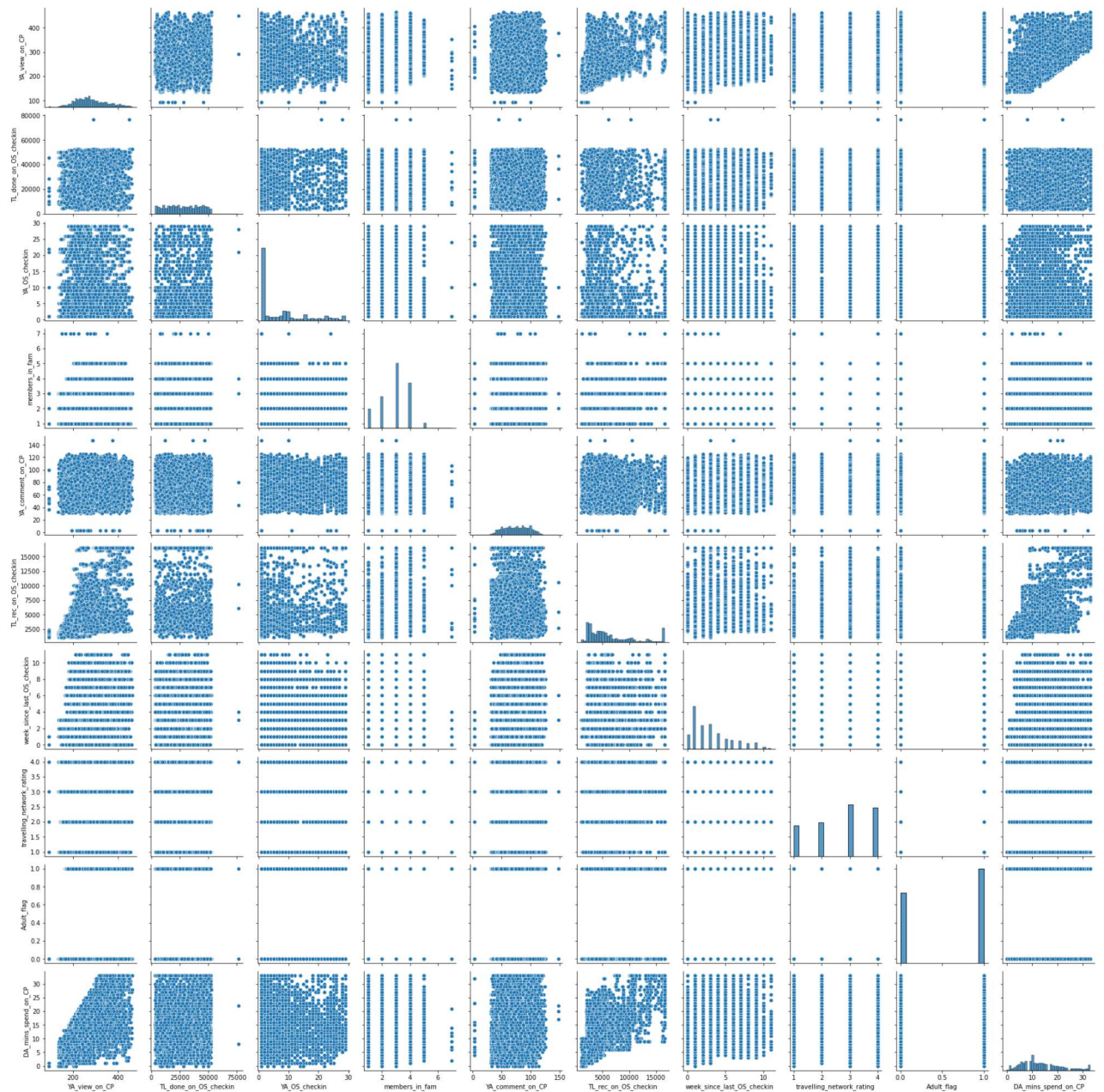
So here in the above heatmap we can see that the light colours shows the highest correlation between 2 variables.

Highest correlated variables are :

1. 'TL_rec_on_OS_checkin' and 'DA_mins_spend_on_CP' (0.72)
2. 'YA_view_on_CP' and 'DA_mins_spend_on_CP' (0.64)
3. 'TL_rec_on_OS_checkin' and 'YA_view_on_CP' (0.5)

```
In [70]: #Pairplot
sns.pairplot(data)
```

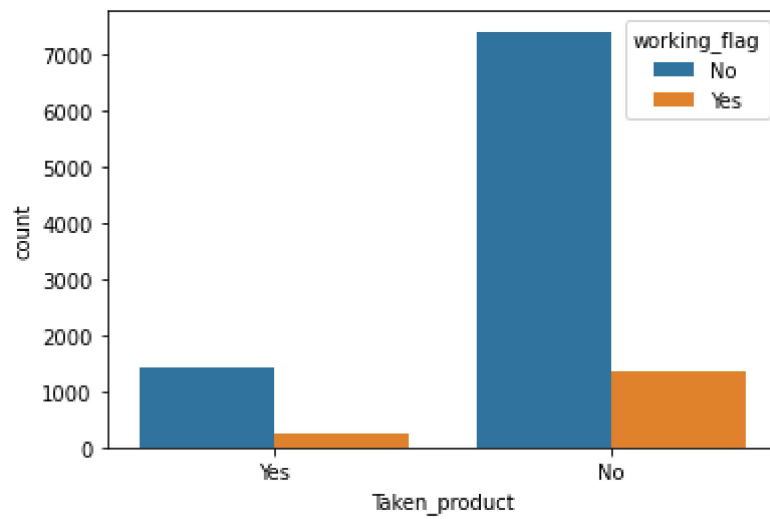
```
Out[70]: <seaborn.axisgrid.PairGrid at 0x176183929a0>
```



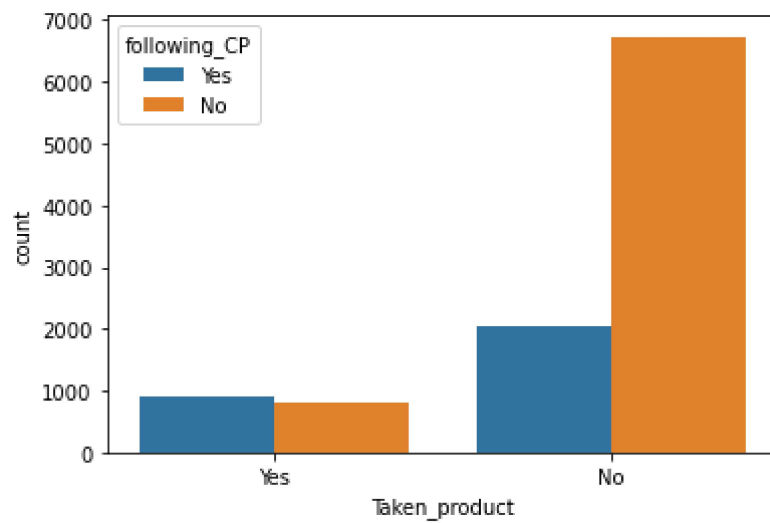
In [71]:

```
#Countplot for Taken product with other variables to compare the relation
```

```
ax = sns.countplot(x="Taken_product", hue="working_flag", data=data)
```



```
In [72]: #Taken_product with following_CP  
  
ax = sns.countplot(x="Taken_product", hue="following_CP", data=data)
```



```
In [ ]:
```